FINITE POSSIBILITY MECHANICS

MODULAR PAPER 04

Tensorless Exact-Ledger Sandbox

Deterministic Integer Policies and Their Boundary with Reference FPM

EXACT-LEDGER PAPER

Alx Spiker
July 2026
Edmonton, Alberta, Canada

Part of the FPM modular research series

Contents

Modular Paper 04 in the Finite Possibility Mechanics research series

Abstract

Tensorless provides a deterministic exact-ledger sandbox for FPM-inspired finite-resource dynamics. Its synchronous sandbox represents energy and action in exact one-third-micro-action subunits, route cost in an unsigned 3×3 integer ledger, and momentum in a separate signed 3×3 ledger. Each tick is a snapshot transaction: reserve action, form payload and momentum proposals, resolve competing capacity claims globally, commit accepted transfers simultaneously, record rejected transfers externally, and close the action-replenishment ledger. The private implementation reports integer-zero energy and momentum residuals for successful ticks; those implementation-specific results are not independently reproducible from this public repository.

Exact ledger closure is not the same claim as exact conformance to every transition in the Python reference. The current Tensorless paper-ledger function allocates the total action globally in proportion to activity weights, whereas Papers 1 and 2 define ordinary one-tick replenishment through the nearest-neighbour kernel $r=P^{T}L$. Tensorless therefore realizes an exact equilibrium-style signed ledger under its own declared contract; it is not yet an exact integer implementation of the complete local transport dynamics.

Three constitutive policies make the continuous-to-discrete contract explicit. R1 defines an integer activity pressure from six-face viscosity contrast and prior-target mismatch. N1 defines reproducible one-quantum antisymmetric route exchanges gated by mobility and paid action. Q1 fixes the normalized route quantum at 10^{-6}. These policies are not numerical patches: they specify how the finite engine resolves choices that a real-valued equation leaves underdetermined. Their status is declared, their ordering is deterministic, and their global invariants are proved below.

> Exact-ledger result
> The declared integer transaction admits exact closure: every accepted unit, rejected unit, starvation deficit, and momentum share occupies an integer account. The associated private implementation report is evidence for that implementation only until a public release or archived source bundle makes independent reproduction possible.

1. Design objective

The Python simulator is the readable reference semantics. Tensorless answers a different question: what additional rules are required to execute the same finite-resource ideas with exact arithmetic, simultaneous contention, deterministic replay, and no tolerance-based conservation decision?

A real-valued specification does not uniquely determine how fractional shares are rounded, how ties are ordered, when action is reserved relative to payload, how competing sources share a nearly full target, or how random route exchange consumes its stream. Tensorless elevates these choices into named operational policies. It is a separate exact policy realization, not an automatic completion of every Python transition: different policies define different finite engines even when they share selected state variables and conservation goals.

1.1 Core and adapter boundary

- The core owns bounded state, transport, ledgers, deterministic scheduling, and diagnostics.

- The caller supplies the integer translation of the complete per-node Lagrangian and payload request.
- Adapters assign physical, computational, chemical, or other domain meaning outside the core.
- The parameter ledger records declared provenance so operational choices cannot disappear into source code.

1.2 Sandbox momentum extension

Tensorless adds a signed integer state $M_i \in Z^{3\times3}$ for payload-flow bookkeeping. This momentum ledger is an adapter-facing extension of the sandbox state; it is not the unsigned route-cost ledger R, is not part of the Paper 2 reference state vector, and is not derived from the five foundational axioms. The core never converts M into energy or action. Action is reserved and closed in its own ledger, while a caller-selected transport fraction determines how much signed M accompanies a payload proposal.

During contention, the transferable signed momentum is partitioned once across the finally accepted payload destinations and one rejected-payload destination. Accepted shares enter target momentum and rejected shares enter the external momentum account. The resulting zero residual proves conservation of this represented integer bookkeeping variable under the sandbox transaction. A physical identification with mechanical momentum requires an adapter-specific units map and correspondence test; it is not established by the integer identity alone.

2. Exact units

$$1 \text{ action} = 1{,}000{,}000 \text{ micro-actions} = 3{,}000{,}000 \text{ sandbox subunits}$$

One sandbox subunit is exactly one third of a micro-action. The normalized FPM energy ceiling 2/3 action is therefore the integer 2,000,000 subunits. The action floor c_0=0.05 action is exactly 150,000 subunits.

$$Q_{action}(c_0) = 3 \times 50{,}000 = 150{,}000 \text{ subunits}$$

> Proposition 1 (Exact representability)
> Every integer micro-action value, the 2/3-action ceiling, and the 0.05-action floor are integers in the one-third-micro-action unit. No rounding is required for these core quantities.

The factor three is a unit conversion, not three separate c_0 charges. This prevents a common accounting error when comparing the exact sandbox with the real-valued reference.

3. Synchronous tick transaction

3.1 Snapshot discipline

All proposals are computed from the same pre-tick snapshot. No source can observe a neighbour's partially committed transfer. This gives the tick an unambiguous simultaneous semantics and removes dependence on loop order.

3.2 Reservation and contention

- Validate state ranges, locks, action floors, and affordability.
- A source with nonzero payload must reserve action first: payload + action ≤ snapshot energy.
- Scatter payload and momentum proposals to the six face slots.
- Resolve all sources competing for each target against its remaining capacity.
- Split momentum once, in proportion to final accepted face energies plus one rejected-energy destination.
- Commit accepted transfers simultaneously and move rejected portions to external ledgers.
- Run exact action debit, replenishment, local spillover, exhaust, and starvation accounting.

3.3 Largest-remainder contention

If a target cannot accept every offer, integer shares are assigned by floor division and remaining units are distributed in this total order: remainder descending; offered payload descending; source flat index ascending; source face ascending. The order is part of the engine definition and guarantees replay across platforms.

> Theorem 1 (Contention conservation)
> Largest-remainder contention assigns exactly min(total offered, target capacity) energy units. No source receives a negative share and no target exceeds capacity.

Proof. Floor shares never exceed the requested total. Each residual unit is assigned only to an open positive-weight claimant, and saturated claimants are removed before recomputation. The loop ends when the accepted sum reaches the lesser of demand and capacity or no capacity remains. QED.

4. Constitutive replenishment R1

R1 converts normalized observables into exact activity pressure without an intermediate integer mean. The ABI field π_i is the integer encoding of the fallback prior π_i used in Papers 1-3, while the ABI field τ_i encodes their local target T_i:

$$W_i=\Sigma_{\text{six faces } f}|\Omega_i-\Omega_{n(i,f)}| + 6|\pi_i-\tau_i|$$

Periodic face multiplicity is retained. The first term is exactly proportional to six times the neighbour-mean viscosity contrast; the second gives the same six-face scale to prior-target mismatch. If all W_i vanish, every weight is set to one, the unique permutation-symmetric positive fallback.

> Proposition 2 (R1 symmetry)
> On a completely homogeneous state, R1 assigns identical positive weight to every node. Relabelling nodes leaves the replenishment allocation unchanged.

The current paper-ledger function first sums the complete action debit and then uses an exact largest-remainder allocator to distribute that total in proportion to W:

$$r_i^{R1} = \mathrm{Allocate}_{\mathrm{integer}}[(\Sigma_j L_j)W_i/(\Sigma_k W_k)]$$

This allocation is global: changing one node's W can change another node's replenishment in the same sandbox tick even when the nodes are not neighbours. R1 therefore defines an exact equilibrium-style closure, not Paper 1's finite-speed microscopic law. The six-face geometry enters the weights, but it does not make the subsequent normalization local. An exact implementation of $r=P^TL$ remains a separate conformance task.

5. Constitutive route exchange N1

N1 represents closed thermal route fluctuations as antisymmetric one-quantum exchanges across edges. Edge slots are visited in flat-node order, then +x,+y,+z, then channel order. Every slot consumes two PCG32 values [4]: an activation roll and a direction bit, even if the exchange is inactive or vetoed.

For each node i, $m_i \in \{0,\ldots,10^6\}$ is the normalized integer mobility observable supplied through the sandbox adapter. Thus $m_i/10^6$ is the node's N1 activation scale. It is a node-level policy input, not automatically the link mobility μ_{ij} of Paper 1; an adapter claiming that correspondence must publish the map from its FPM mobility calculation to m_i.

$$m_{\mathrm{edge}}=\min(m_A,m_B), \quad \mathrm{cutoff}=\mathrm{floor}(m_{\mathrm{edge}}2^{32}/10^6)$$

$$\text{exchange iff uint32\_roll} < \mathrm{cutoff}$$

An activated exchange subtracts one Q1 unit from one endpoint's channel and adds one to the other. It is vetoed at either route boundary, on a self-loop, or unless both endpoints supplied and could afford at least one c_0 action floor for the tick. N1 belongs to the already-paid route-candidate-evaluation operation; it does not add a separate fee per trial.

> Theorem 2 (N1 route-sum conservation)
> For every channel, every accepted N1 exchange preserves the global unsigned route-cost sum exactly.

Proof. One endpoint changes by -1 and the other by +1 in the same channel. Vetoed exchanges change neither. Summing over any sequence of trials gives zero net change. QED.

6. Route quantization Q1

$$\mathrm{policy\_scale}=1{,}000{,}000, \quad q_R=1/\mathrm{policy\_scale}=10^{-6}$$

Q1 fixes the representable normalized route interval to integer values 0 through 1,000,000. It is the lattice spacing of the route ledger in the constitutive modes. Its value determines resolution, state count, and N1 trial probability; changing it defines a different exact realization and must change the parameter fingerprint.

> Proposition 3 (Route bounds)
> N1 plus its boundary veto leaves every normalized route channel in the closed integer interval [0,policy_scale].

7. Exact conservation identities

7.1 Energy

$$E_{initial}=E_{current}+E_{external}-E_{starvation}$$

External exhaust contains energy removed from represented nodes by rejected or unrouteable transfers. Starvation is subtracted because clipping an unaffordable negative raw balance to zero raises stored energy relative to the ideal signed update. The signs match the expanded ledger proved in Paper 1. Tensorless computes the originating node event during the transaction but exposes cumulative aggregate exhaust and starvation counters; it does not currently retain the Python runtime's full site-by-tick event history.

> Theorem 3 (Integer energy closure)
> If a sandbox tick returns success, the energy identity holds with integer residual exactly zero and every node satisfies $0 \le E_i \le E_{ceiling}$.

7.2 Momentum

$$M_{initial,c}=M_{current,c}+M_{external,c} \quad \text{for each of nine channels } c$$

Accepted momentum enters the target; rejected momentum enters the external account. Quantizing only after contention ensures accepted and rejected shares partition the source's transferable momentum once, avoiding compounded rounding.

> Theorem 4 (Integer momentum closure)
> If a sandbox tick returns success, all nine signed momentum identities have residual exactly zero.

7.3 Thermal noise boundary

Independent additive momentum noise is excluded from the exact sandbox because no closed paper-specified noise ledger currently balances it. N1 supplies a closed route fluctuation, not a momentum-noise substitute. This preserves exactness instead of hiding an unbalanced random term behind tolerance.

8. Parameter provenance

Every declared run parameter can be classified FIXED, DERIVED, FREE, or FITTED. The parameter ledger records name, full-precision value, class, and source; sorts entries canonically; and computes a 64-bit FNV-1a drift fingerprint over core identity, hashes, and entries.

- FIXED: selected before the run and held constant.
- DERIVED: computed from fixed inputs.
- FREE: selected by caller or adapter without fitting to the evaluated data.
- FITTED: adjusted against data.

The fingerprint detects declared drift; it is intentionally non-cryptographic and does not replace archival hashes or signatures. Its scientific purpose is to make parameter changes visible and to distinguish a fixed constitutive policy from a fitted adapter coefficient.

9. Correspondence with the Python reference

The two runtimes share the finite cubic geometry, 3×3 route ledger, bounded energy, signed starvation and exhaust accounts, finite carrier concepts, and local-capacity routing. They differ in representation, scheduling, retained diagnostics, and operational kernels. Correspondence must therefore be claimed contract by contract.

- Python evaluates real-valued equations directly; Tensorless executes integer transactions.
- Python ordinary replenishment uses the local reversible $P^{T}L$ kernel; the Tensorless paper-ledger function and R1 modes use exact proportional allocation under their declared contract.
- Python phase evolution uses complex arithmetic; Tensorless stores scaled carrier and route state.
- Tensorless defines contention order, random-stream consumption, action reservation, and momentum partition exactly.
- Python retains site-resolved exhaust, starvation, and Landauer event increments; Tensorless currently exposes cumulative aggregate counters.
- Python derives its Lagrangian and carrier transitions internally; the Tensorless sandbox receives caller-translated action and payload values.

> Cross-runtime audit target
> Agreement is required only for a named shared contract and an explicit continuous-to-discrete input map. Exact zero residual in Tensorless proves its represented ledger identity; it does not by itself prove trajectory equivalence, the local kernel, carrier equivalence, or a physical interpretation. Differences caused by R1, N1, or Q1 are policy effects to measure, not floating-point errors to conceal.

10. Conclusion

Tensorless demonstrates that an FPM-inspired finite-resource transaction can be executed with exact conservation, bounded state, deterministic contention, and reproducible stochastic route exchange. The additional rules are visible: one-third-micro-action units make the core scales integral; R1 defines a global activity-weighted allocator; N1 defines paid, conservative route fluctuation; and Q1 defines route resolution. Their explicitness turns implementation ambiguity into a testable model family while leaving local-kernel conformance as an open engineering obligation.

Exact-ledger summary

Tensorless does not merely round the Python simulator. It defines a distinct exact transaction whose represented energy and momentum ledgers are specified to close identically. The private implementation report is strong but not independently checkable in this publication set; the current sandbox is also not yet the exact-integer form of every FPM transition.

References

[1] A. Spiker, Tensorless Engine: Experimental Synchronous FPM Sandbox, private source documentation and implementation, files `docs/fpm-sandbox.md`, `core/src/tensorless_sandbox.cpp`, and `core/src/tensorless_ledger.cpp` (accessed July 2026). Implementation-specific claims citing this source are not independently checkable until an immutable release, commit hash, or archival bundle is made available.

[2] A. Spiker, FPM Foundations, Modular Paper 01.

[3] A. Spiker, Executable FPM, Modular Paper 02.

[4] M. E. O'Neill, "PCG: A Family of Simple Fast Space-Efficient Statistically Good Algorithms for Random Number Generation," Harvey Mudd College technical report HMC-CS-2014-0905 (2014). https://www.pcg-random.org/paper.html